

Preguntas de reflexión

- ¿Qué problemas genera usar múltiples condicionales para estados?

Usar muchos if, else o switch genera varios problemas:

- La clase principal crece demasiado.
- Las reglas de negocio quedan mezcladas.
- Es más difícil agregar nuevos estados.
- Aumenta el riesgo de errores.
- Se viola el principio de responsabilidad única.
- Probar cada combinación de condiciones se vuelve complicado.

Por ejemplo:

```
/*
 * Forma genera usar múltiples condicionales para estados
 */
if (status == PENDING) {
    if (operation.equals("confirm")) {
        // ...
    } else if (operation.equals("cancel")) {
        // ...
    }
} else if (status == CONFIRMED) {
    // ...
}
```

- ¿Cómo facilita Builder la creación de reservas complejas?

Builder permite crear la reserva paso a paso y solamente agregar las características necesarias:

```
/*
 * Builder permite crear la reserva paso a paso y solamente agregar las características necesarias
 */
new ReservationBuilder()
    .withPassenger(passenger)
    .withFlight(flight)
    .withSeat("12A")
    .withSeatClass(SeatClass.PREMIUM)
    .addService(AdditionalService.BAGGAGE)
    .withPreferences("Asiento de pasillo")
    .withPricingStrategy(new PremiumPricing())
    .build();
```

Además, puede validar campos obligatorios antes de devolver el objeto final.

- ¿Qué ocurre si cambian las reglas de precio?

Si las reglas cambian, se puede crear una nueva estrategia o modificar una estrategia existente sin tocar la clase Reservation.

Por ejemplo, una nueva estrategia para temporada alta:

```
/**
 * Estrategia para temporada alta.
 */
public class HighSeasonPricing implements PricingStrategy {
    @Override
    public double calculatePrice(Reservation reservation) {
        double price = reservation.getBasePrice() * 1.25;

        for (AdditionalService service :
            reservation.getAdditionalServices()) {
            price += service.getPrice();
        }

        return price;
    }
}
```

Luego se utiliza así

```
reservation.setPricingStrategy(
    new HighSeasonPricing()
);
```

- ¿Cómo desacoplar notificaciones?

Las notificaciones se desacoplan utilizando el patrón Observer.

La clase Reservation actúa como sujeto o emisor de eventos, mientras que las clases EmailNotifier, SMSNotifier y AppNotifier actúan como observadores. Reservation no necesita conocer cómo se envía cada notificación; únicamente informa que ocurrió un evento.

```
/*
 * 3. Registrar observadores.
 */
reservation.addObserver(
    new EmailNotifier(passenger.getEmail())
);

reservation.addObserver(
    new SMSNotifier(passenger.getPhone())
);

reservation.addObserver(
    new AppNotifier(passenger.getId())
);
```

Cuando cambia el estado de la reserva, se notifica a todos los observadores:

```
Windsurf: Refactor | Explain
public class ConfirmedState implements ReservationState {
    /* La reserva ya está confirmada */
    Windsurf: Refactor | Explain | X
    @Override
    public void confirm(Reservation reservation) {
        System.out.println(
            x: "La reserva ya está confirmada."
        );
    }
}
```

Esto permite agregar nuevos canales, como WhatsApp o notificaciones push, sin modificar la clase Reservation.

- ¿Qué patrón mejora cohesión en cambios de estado?

Cada clase representa un estado específico de la reserva y contiene únicamente las reglas relacionadas con ese estado:

- PendingState
- ConfirmedState
- CancelledState
- CheckedInState
- BoardedState

Por ejemplo, PendingState sabe cómo confirmar o cancelar una reserva pendiente, mientras que CancelledState sabe que una reserva cancelada no puede modificarse ni confirmarse.

```
/* Se pasa al estado de confirmada */
Windsurf: Refactor | Explain | X
@Override
public void confirm(Reservation reservation) {
    reservation.changeState(new ConfirmedState());
}

/* Se pasa al estado de cancelada */
Windsurf: Refactor | Explain | X
@Override
public void cancel(Reservation reservation) {
    reservation.changeState(new CancelledState());
}
```

Así se evita colocar numerosos if, else o switch dentro de Reservation. Cada estado mantiene juntas sus reglas relacionadas, aumentando la cohesión y facilitando agregar nuevos estados sin afectar demasiado el código existente.